> **Actualización A01–A18 · 01 nov 2026.** Modelo estrella, soporte, contratos, compuertas, CI y herramientas locales implementados. Los diagnósticos originales se conservan como antecedente; su cierre vigente y evidencias están en Estado de implementación. SQL Server, DAX y Delta cloud siguen pendientes.

ANDINA MARKET / DATA & DECISIONS

Una cifra.
Muchas decisiones.
La misma confianza.

De archivos con errores a una base trazable para dirigir el negocio.

PRUEBA TÉCNICA · LIDERAZGO DE DATOS

14.177 millones COP

Venta neta del caso · enero 2025 — septiembre 2026
Valor exacto: COP 14.177.333.600

Manuel Alberto Coy Benavides

Revisión del proyecto · 01 NOV 2026

16 capítulos + diagnóstico

01 / EL PUNTO DE PARTIDA

Cuando cada área tiene su cifra, *la decisión pierde fuerza.*

El caso parte de SQL Server, más de 30 informes de Power BI y fuentes que todavía dependen de Excel.

01

Fuentes dispersas

Pedidos, pagos, clientes y soporte.

02

Definiciones distintas

Fechas, estados y duplicados cambian el resultado.

03

Reportes manuales

Cada equipo reconstruye su propia verdad.

04

Decisiones frágiles

Se discute la cifra antes de actuar.

CONTRATO

Qué cuenta

Bruto − descuento, solo pedidos ENTREGADO, atribuidos a fecha_pedido.

TRAZABILIDAD

De dónde viene

Cada resultado debe volver al registro de origen y a la regla aplicada.

GOBIERNO

Quién responde

Finanzas y Comercial validan la definición; un dueño aprueba los cambios.

LECTURA CLAVE

La solución une tres cosas: una definición explícita, un proceso repetible y una responsabilidad de negocio.

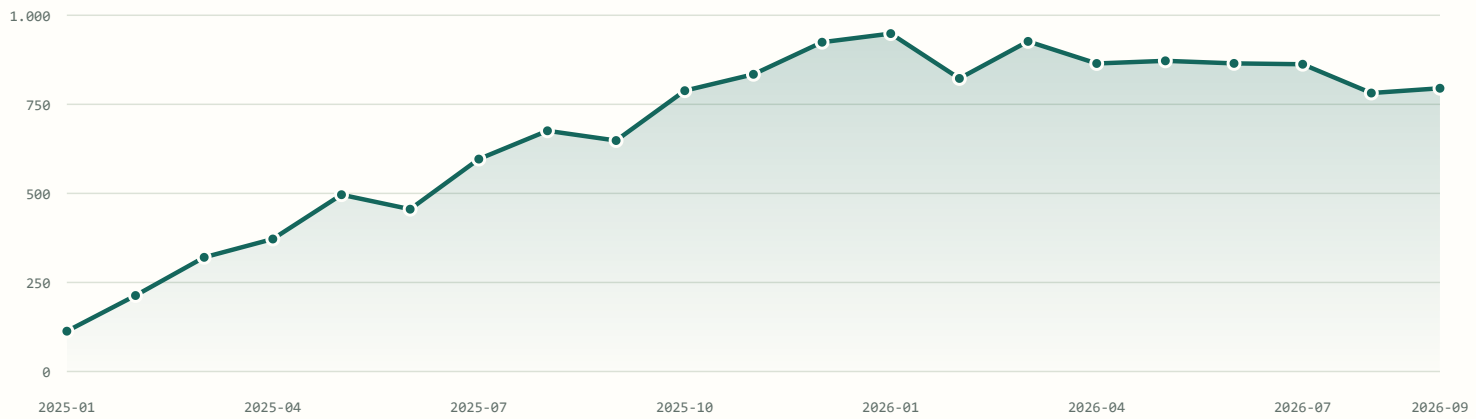
02 / LO QUE DICEN LOS DATOS

La venta se estabilizó.
La mezcla cambió.

Explora los 21 meses de venta neta calculados desde Gold. Valores en millones de pesos colombianos.

Total APP WEB TIENDA

Consulta local · 01 nov 2026



TOTAL · 2026-09 · COP 795,1 millones

APP · JUN → SEP 2026

+19,2 %

De $376,5 M a $448,9 M. El crecimiento corresponde a este intervalo concreto.

TIENDA · JUN → SEP 2026

−51,4 %

De $217,6 M a $105,8 M. Una señal prioritaria para investigar.

TOTAL · SEP 2026

$795,1 M

La comparación anual positiva no elimina el estancamiento reciente.

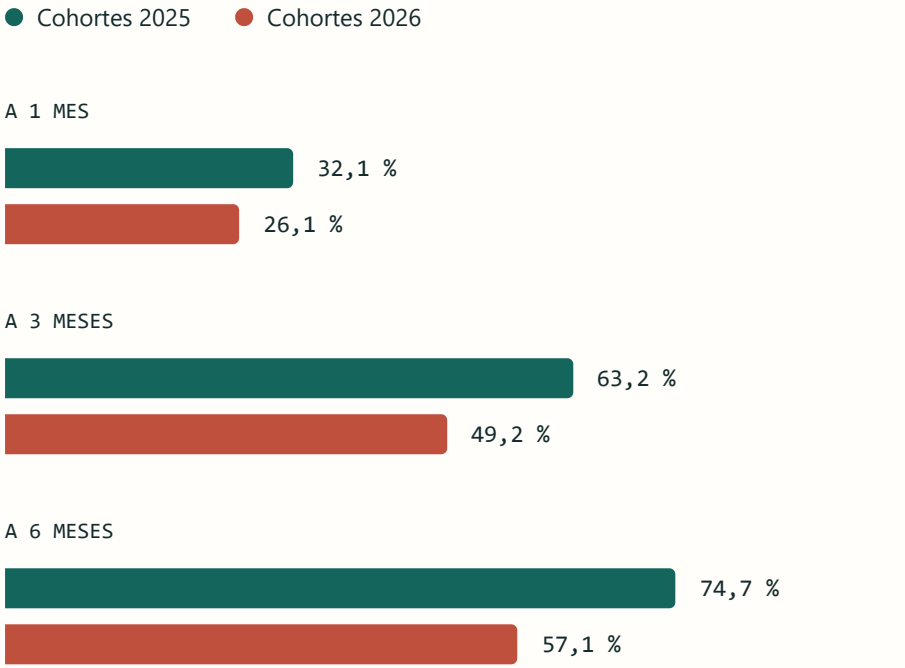
LECTURA CLAVE

Primera decisión: investigar TIENDA por ciudad, categoría y operación. No atribuir la caída a una causa sin contrastarla.

Evidencia · Consulta de solo lectura a data/warehouse.duckdb · 63 filas mensuales · Lectura SQL

Vender hoy no garantiza *que vuelvan mañana.*

Recompra de personas con un primer pedido entregado y ventana completa. Cohortes agrupadas por año de registro.



Ventanas de 1, 3 y 6 meses calendario desde el primer pedido. A 6 meses, 2026 solo tiene 856 personas maduras de enero a marzo.

−14,0

puntos porcentuales a 3 meses

La tasa pasa de 63,2 % a 49,2 %. El deterioro persiste usando todos los registrados como base: 60,9 % → 46,4 %.

La identidad cambia el análisis.
6.040 registros de clientes representan 6.000 personas al normalizar el documento.

01

Cohorte

Mes de registro de la persona.

02

Primera compra

Primer pedido entregado.

03

Ventana madura

Tiempo suficiente para observar.

04

Recompra

Otro pedido en fecha posterior.

LECTURA CLAVE

TIENDA creció 59,8 % de junio a septiembre de 2025 y cayó 51,4 % en 2026. El patrón no se repite; dos años no bastan para descartar estacionalidad.

Evidencia · Recompra y denominadores · SQL · contraste de TIENDA consultado en Gold

04 / CONSTRUIR CONFIANZA

Los errores no desaparecen.
Se vuelven visibles.

15 tipos de problemas documentados. El tratamiento depende de si el dato se puede corregir, conservar marcado o aislar.

PEDIDOS

19.734 recibidos



19.270 válidos 464 aislados · 2,35 %

PAGOS

20.883 recibidos



20.124 válidos 759 aislados · 3,63 %

01 · NORMALIZAR

Sin perder el original

810 canales con variantes; 1.200 fechas de pago con dos formatos; documentos que ocultan 40 duplicados.

02 · CONSERVAR Y MARCAR

No borrar una venta real

60 pedidos con cliente inexistente permanecen con una bandera de calidad.

03 · AISLAR

Con motivo y contexto

Duplicados, valores inválidos y fechas incoherentes van a cuarentena con la fila original.

122 pedidos entregados sin pago registrado

345 pagos asociados a pedidos rechazados

LECTURA CLAVE

Estos hallazgos se escalan a negocio. No permiten concluir automáticamente que hay fraude, cartera vencida o doble cobro.

Evidencia · Catálogo de calidad · Reglas · conteos de referencia; una fila puede incumplir varias reglas.

05 / DEL ARCHIVO AL INDICADOR

Una cadena de evidencia.
De principio a fin.

Implementación local: Python + DuckDB. Bronze conserva el origen; Silver aplica reglas; Gold entrega una cifra utilizable.

01
CSV
Clientes · pedidos · pagos

02
Bronze / Parquet
Snapshot crudo · SHA-256 · run_id

03
Silver / DuckDB
Tipos · deduplicación · row_hash

04
Gold / DuckDB
Día × canal × categoría

↳ **Cuarentena**
Registro + regla + hash + JSON original

↳ **Control**
Watermark + historial de ejecuciones

↳ **Consumo**
SQL analítico · reporte · modelo propuesto

TRANSACCIÓN DUCKDB · Silver + cuarentena + watermark + Gold se confirman juntos

TRAZABILIDAD

Un archivo, una huella

El manifiesto evita aterrizar dos veces el mismo contenido; Silver lee la copia Bronze.

GRANO ANALÍTICO

7.925 filas Gold

16.855 pedidos entregados agregados por fecha, canal y categoría.

PORTABILIDAD

Reglas en PySpark

Existe implementación paralela y notebook Delta. Ejecución en nube todavía pendiente.

LECTURA CLAVE

El agregado diario y el modelo estrella local ya existen y concilian. El modelo semántico y la carga de la plataforma requieren validación propia.

Evidencia · pipeline.py · bronze.py · store.py

06 / LO QUE PASA CUANDO ALGO CAMBIA

Volver a ejecutar
debe ser seguro.

Explora cómo responde el diseño a cuatro situaciones. Son explicaciones del código, no ejecuciones del ETL desde esta página.

COMPORTAMIENTO DEL DISEÑO

Sin cambios, sin duplicados

`Mismo SHA-256 → mismo Bronze → mismo row_hash`

El manifiesto reutiliza el snapshot. Silver no reescribe filas idénticas; la llave de cuarentena evita repetir incidencias.

Resultado esperado: 0 insertados y 0 actualizados.



LECTURA CLAVE

Pedidos: lectura completa y escritura por hash. Pagos: filtro por fecha_carga más tres días, después de leer el snapshot. Es incremental en procesamiento/escritura, no una extracción CDC de origen.

Evidencia · Pruebas de idempotencia, cambios y rollback · Límites del incremental

07 / UNA DEFINICIÓN QUE VIAJA

El mismo contrato *en cada pantalla.*

El SQL, el agregado diario y las medidas DAX deben reconciliarse. La definición está escrita; falta certificarla en el motor destino.

CONTRATO DEL CASO

Σ (bruto − descuento)

`estado = ENTREGADO`

Ticket promedio = venta neta / pedidos entregados

Fecha de atribución: fecha_pedido.
2025-01-01 → 2026-09-30.



COMPARACIÓN CORRECTA

Ene–sep vs ene–sep

Para 2026: $7.739,1 M frente a $3.891,3 M en 2025 → +98,88 %.

COMPARACIÓN ENGAÑOSA

9 meses vs 12 meses

Usar todo 2025 como base daría +20,2 %. El período comparable importa.

SIN BASE ANTERIOR

Vacío, no cero

En 2025 no hay año previo disponible. DAX debe devolver BLANK.

LECTURA CLAVE

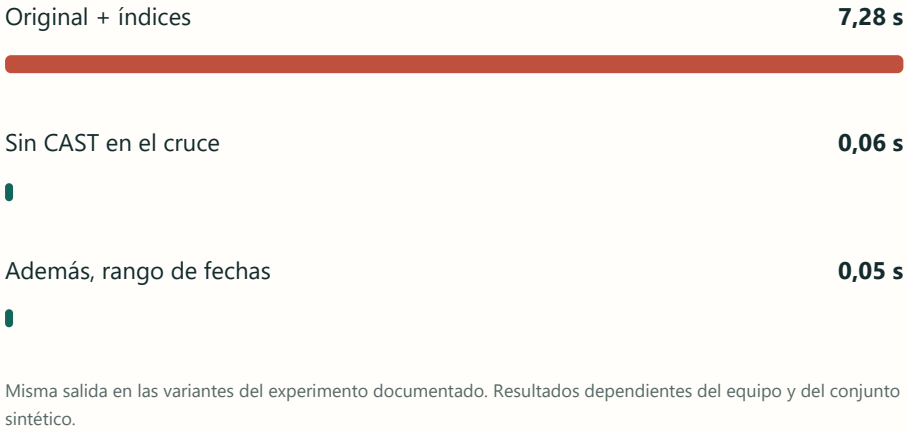
Brecha crítica: el ETL local genera el agregado diario, pero no carga el esquema estrella del DDL. Falta construir ese puente antes de validar DAX y el reporte ODBC.

Evidencia · medidas.dax · Valores esperados · DDL

El índice no arregla *una búsqueda que lo ignora.*

Experimento documentado en SQLite con datos sintéticos. Evidencia sobre la forma del problema, sin extrapolar tiempos a SQL Server.

ABLACIÓN · 100.000 PEDIDOS · SQLITE



Misma salida en las variantes del experimento documentado. Resultados dependientes del equipo y del conjunto sintético.

DEL FILTRO A LA BÚSQUEDA

```
YEAR(fecha_pedido) = ...
CAST(t.id_cliente AS VARCHAR) = ...

↓

fecha_pedido >= @inicio
AND fecha_pedido < @fin
t.id_cliente = c.id_cliente
```

01

Agregar una vez

Evitar trabajo correlacionado repetido.

02

Filtrar por rango

Conservar la columna de fecha sin funciones.

03

Cruzar tipos iguales

Eliminar conversiones innecesarias.

04

Validar en destino

Plan real · IO · CPU · equivalencia.

LECTURA CLAVE

Pendiente: benchmark representativo en SQL Server, incluidos costo de escritura de índices, sesgo de datos y parámetros. No hay prueba de rendimiento sobre 50 millones en ese motor.

09 / ARQUITECTURA OBJETIVO

Escalar la confianza, *no la complejidad.*

Propuesta a 12 meses del proyecto. Fabric es la hipótesis inicial por la base existente de Power BI; la decisión requiere piloto y cotización.

FUENTES

SQL Server on-premise

E-commerce · App · POS

Soporte

↓ conectividad y contratos ↓

01
Ingesta
Gateway / APIs · incrementos

02
Lakehouse
Bronze → Silver · reglas

03
Publicación
Gold · modelo de consumo

04
Power BI
Modelo semántico · informes

GIT + CI/CD · DEV / TEST / PROD · CATÁLOGO Y LINAJE · SECRETOS Y PERMISOS · MONITOREO

HIPÓTESIS PREFERIDA

Microsoft Fabric

Probar el recorrido completo con los informes prioritarios y medir operación, conciliación y costo. La experiencia del equipo en SQL/BI es un supuesto por validar.

ALTERNATIVA

Databricks

Comparar con la misma carga si ingeniería, ML, streaming o requisitos multicloud justifican otra plataforma. No decidir solo por volumen.

LECTURA CLAVE

La prueba local reduce incertidumbre sobre reglas. No prueba conectividad, permisos, MERGE Delta, capacidad, costos ni operación de ninguna nube.

Evidencia · Arquitectura original de dos páginas · Notebook generado · sin afirmaciones de precios o licencias vigentes.

Una carga puede terminar.
Y aun así no estar lista.

La transacción evita estados parciales. La conciliación local ya bloquea publicación; los umbrales operativos requieren aprobación.

Carga terminada

Control técnico

↓

¿Concilia?

Volumen · calidad · frescura

↓

SÍ → Publicar versión validada

NO → Retener + alertar + investigar

ANTES DEL CAMBIO

PR → pruebas → Test

Versionar DDL, notebooks, reglas y modelo. Proteger la promoción a producción.

DURANTE LA CARGA

Medir y reconciliar

Definir frescura, duración, filas, rechazos, totales y umbrales con historial real.

DESPUÉS DEL FALLO

Recuperar y comunicar

Runbook, responsable, última versión válida y ensayo de restauración.

LECTURA CLAVE

Ya existen CI, compuerta local y respaldo/restauración a destino nuevo. Faltan programación, alertas, permisos, retención y recuperación cloud. Un git revert revierte código; no revierte automáticamente datos, esquemas ni modelos publicados.

Evidencia · Diseño de operación · Registro local de corridas · diagnóstico de archivos disponibles.

Clasificar es fácil.
Demostrar valor, no tanto.

El experimento de tickets ya tiene resultados guardados. Su principal límite es la independencia y diversidad de la evaluación.

01

2.125 tickets

Datos del caso

02

34 textos distintos

Una llamada por texto

03

17 tipos + otro

Salida JSON validada

04

Evaluación

Reglas vs dos LLM

EXACTITUD · 150 FRASES SINTÉTICAS



Resultados documentados para DeepSeek Flash y V4 Pro. Las reglas se ajustaron tras observar errores: el conjunto dejó de ser totalmente independiente.

LO QUE ESE 100 % NO PRUEBA

Desempeño en tickets nuevos reales

Las etiquetas fueron propuestas por IA y siguen pendientes de revisión humana. Los dos modelos empatan en un conjunto que puede ser demasiado fácil.

LECTURA CLAVE

Siguiente puerta: etiquetas humanas, muestra real reservada, F1 por clase, revisión de errores críticos y costo por ticket. El saldo medido de ~USD 0,03 cubre parte de las evaluaciones; no es un precio unitario.

La confianza también *se opera con personas.*

Código revisable, incidentes comunicados y estándares que el equipo puede cumplir.

07:30

Contener

Marcar la cifra en revisión y pausar distribución.

08:00

Diagnosticar

Contrastar fuente, Gold, carga y últimos cambios.

09:00

Corregir

Reprocesar o revertir; conciliar antes de enviar.

09:30

Confirmar

Informar a los mismos destinatarios con evidencia.

Secuencia propuesta para el incidente del enunciado; no es un tiempo de recuperación medido.

REVISIÓN DEL PR

Corregir y enseñar

Ocho bloqueantes documentados: secretos, inyección SQL, estado mutable, definición errónea de ventas, filtro de fecha, errores ocultos, API obsoleta y división por cero. Versión corregida con cinco pruebas.

ESTÁNDARES DEL EQUIPO

Acordar y sostener

Conversación privada, revisión de fallos, rol activo del experto y 30 días de seguimiento. PR, revisión y pruebas como mínimos comunes.

LECTURA CLAVE

El reporte ODBC sigue pendiente en destino. Se añadió vista previa local idempotente, sin envío de correo.

90 días para ganar confianza.
12 meses para extenderla.

Hoja de ruta propuesta. Cada fase termina con evidencia aceptada, no solo con componentes instalados.

01–30

Acordar y conectar

Inventario de informes, contrato firmado, dueños por dominio, repositorio y ambientes.

Salida: definición aprobada + ingesta trazable.

31–60

Limpiar y conciliar

Silver, cuarentena, carga del modelo destino y Gold contra el transaccional.

Salida: diferencia cero o explicada y aceptada.

61–90

Certificar y adoptar

Validar DAX, migrar informes prioritarios, activar alertas y practicar recuperación.

Meta propuesta: 8 informes en el modelo oficial.

Meses 4—6

Más fuentes y dominios

Meses 7—9

Migración del resto de informes

Meses 10—12

Gobierno, ML e IA con evidencia

LECTURA CLAVE

El primer hito no es “elegir una nube”. Es lograr que Finanzas y Comercial acepten la misma cifra y puedan rastrear cualquier diferencia.

Evidencia · Plan original · metas y responsables propuestos, sujetos a capacidad y presupuesto.

Una demostración sólida *tiene límites explícitos.*

El estado se basa en código, documentación, resultados existentes y una nueva ejecución de las pruebas locales.

Verificado en esta revisión

Base local

- Suite local ampliada; conteo y omisiones en el registro de evidencia vigente.
- Consulta de 63 agregados mensuales de Gold.
- Contraste APP y TIENDA, junio–septiembre.
- Inspección de ETL, SQL, DAX, reporte y cliente LLM.

Documentado en el proyecto

Ejecuciones previas

- Paridad de reglas Spark con DuckDB.
- Recálculo independiente con pandas.
- Experimentos de optimización en SQLite.
- Evaluación LLM con archivos de resultados.

Sin validación en destino

Plataforma real

- T-SQL, DDL y reporte ODBC en SQL Server.
- Medidas y relaciones en Power BI.
- MERGE, DELETE y recuperación Delta en nube.
- Operación, seguridad y costo del servicio.

La documentación también necesita reconciliación.

README y lectura SQL declaran Spark SQL ejecutado; P02 sigue “no confirmado”. La arquitectura cita 90 pruebas y otros archivos 85/89 según entorno. La entrega figura el 31 de octubre frente al límite del 21. La página anterior decía 20 pendientes, pero el inventario llega a P21.

LECTURA CLAVE

Una afirmación solo cambia de estado cuando existe un resultado reproducible asociado a fecha, entorno y artefacto. No presentar “escrito” como “desplegado”.

Evidencia · Inventario original P01–P21 · Pruebas y consulta de esta revisión

La base está construida.
Ahora hay que cerrar el circuito.

El salto a producción depende de integración, evidencia y operación. El diagnóstico completo está en el siguiente capítulo.

01 definición compartida

PRIMERO

Cerrar la entrega

Unificar estados y fechas, conservar evidencias, revisar etiquetas y confirmar las acciones de seguridad pendientes.

DESPUÉS

Validar el destino

Construir la carga del modelo estrella y ejecutar T-SQL, DAX, ODBC y Delta en el entorno elegido.

PARA OPERAR

Convertirlo en servicio

CI/CD, compuerta de publicación, permisos, monitoreo, recuperación y adopción por negocio.

La promesa no es tener más datos.
Es poder *decidir con ellos*.

ANEXO / DIAGNÓSTICO ACCIONABLE

Diagnóstico inicial.
Avances con evidencia.

21 pendientes del inventario original + 12 brechas de integración y operación identificadas al leer el proyecto. No todos bloquean la prueba técnica; varios pertenecen al paso a producción.

Prioridad sugerida

Primero: coherencia documental, evidencia y acciones de seguridad aplicables. Después: modelo destino y validación funcional. Antes de producción: compuertas, permisos, recuperación y operación. D01–D12 son diagnóstico adicional; P01–P21 conservan el texto original con notas de actualización donde corresponde.

P01 Pruebas de paridad Spark frente a DuckDB

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: confirmado en nuestra revisión con Python 3.11 y PySpark 3.5.3 (89 pasan). Falta guardar la salida desde tu equipo.

Qué hacer:
1. Crea un entorno con Python 3.11 (con 3.12 las pruebas de Spark se omiten o fallan en Windows). Instala `requirements.txt`, `requirements-spark.txt` y Java 17.
2. Desde la raíz del repositorio, ejecuta:
- `python -m pytest tests/test_spark_paridad.py -v`
- `python src/shared/spark/verificar_paridad_real.py`
3. Guarda ambas salidas.

Criterio de cierre: las pruebas de paridad pasan (sin omisiones) y el script imprime `PARIDAD COMPLETA`, con las cinco cifras y los nueve conteos por regla iguales a DuckDB.

Evidencia: `pytest_paridad.txt` y `verificar_paridad_real.txt`.

P02 Spark SQL igual a DuckDB en las consultas de venta

> **Revisión 01 nov 2026**
>
> Hay contradicción con README/01_sql_lectura. La suite actual omite Spark; no se cierra la validación.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no confirmado. En Windows falla por falta de `HADOOP_HOME` con `winutils.exe`.

Qué hacer:
- Opción A, Windows: instala `winutils.exe` compatible con Hadoop 3 desde una fuente que confíes, define `HADOOP_HOME` y ejecuta `python -m pytest tests/test_sql_spark.py -v`.
- Opción B, Linux o WSL: crea el entorno ahí y ejecuta el mismo comando.
- Requisito: que exista `data/warehouse.duckdb` (lo genera `python scripts/cargar_datos.py --desde-cero`).

Criterio de cierre: la prueba pasa sin omitirse.

Evidencia: `pytest_sql_spark.txt`.

P03 T-SQL de venta neta y recompra en SQL Server

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no ejecutado. Antes se afirmó "verificado por traducción mecánica", pero no hay ningún artefacto que lo respalde. Esa afirmación se retiró.

Qué hacer:
1. Instala SQL Server Developer (gratuito) o usa Docker con la imagen oficial de SQL Server.
2. Completa P04 primero: crea el modelo con `sqlserver/ddl_tsql.sql`.
3. Carga las tablas de Silver (clientes, pedidos, pagos) desde DuckDB: exporta a CSV con consultas sobre `data/warehouse.duckdb` y usa `BULK INSERT` o el asistente de importación.
4. Ejecuta `sqlserver/01_ventas_recompra.sql` completo.
5. Compara con las cifras de referencia.

Criterio de cierre:
- La consulta de venta neta mensual por canal devuelve 63 filas.
- La suma de la venta neta es 14.177.333.600.
- La recompra 2026 a 1, 3 y 6 meses es 26,1 %, 49,2 % y 57,1 %.

Evidencia: la salida de las consultas (exportada a CSV o captura), la versión de SQL Server (`SELECT @@VERSION`) y el `resultado.md`.

P04 DDL del modelo en SQL Server o Fabric Warehouse

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no ejecutado.

Qué hacer:
1. Crea una base de datos vacía en SQL Server.
2. Ejecuta `sqlserver/ddl_tsql.sql` por lotes (el archivo usa `GO` como separador).
3. Revisa si algún lote falla. Si una clave foránea o una longitud de columna falla, anota el error exacto y no lo ocultes.

Criterio de cierre: el script se ejecuta sin errores. Si hay errores, quedan documentados y se corrigen en el repositorio con una nota.

Evidencia: el log completo de ejecución y el listado de tablas creadas (`SELECT name FROM sys.tables`).

P05 Medidas DAX en Power BI Desktop

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no ejecutado. Los valores esperados salen de un cálculo en SQL.

Qué hacer:
1. Instala Power BI Desktop (gratuito, Windows).
2. Importa los datos de Gold y las tablas que necesitan las medidas. Ojo: el archivo `fabric/semantic_model/medidas.dax` espera una tabla `fact_pedido` (un pedido por fila, con `fecha_key`, `estado`, `valor_bruto`, `descuento`) y una `dim_fecha` con una columna `fecha` marcada como tabla de fechas. El almacén local no tiene `fact_pedido` ni `dim_fecha` tal cual: constrúyelas desde `silver_pedidos` y un calendario de 2025 a 2026, y documenta la construcción.
3. Crea la relación entre `fact_pedido[fecha_key]` y `dim_fecha[fecha_key]`.
4. Agrega la columna calculada `tiene_datos` y las medidas de `medidas.dax`.
5. Compara los cinco meses de la tabla de `docs/03_dax.md` (enero, febrero, marzo, junio y septiembre de 2026).
6. Compara también la variación anual de 2026 enero a septiembre: debe dar +98,9 % con la limitación de fechas, y +20,2 % sin ella.

Criterio de cierre: venta, pedidos, ticket y variación coinciden con la tabla de valores esperados (margen de redondeo de un punto porcentual).

Evidencia: captura de la tabla con las medidas y el archivo `.pbix` (sin datos de clientes; si contiene datos personales, no lo subas al repositorio).

P06 Notebook Delta en Fabric

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no ejecutado. Incluye MERGE, DELETE e historial de Delta.

Qué hacer:
1. Crea un espacio de trabajo en Microsoft Fabric con la prueba gratuita o una capacidad disponible (verifica la oferta vigente).
2. Crea un Lakehouse y sube los cuatro CSV a `Files/andina/raw/`.
3. Importa `fabric/notebooks/andina_etl_spark.ipynb`, adjunta el Lakehouse y deja `PLATAFORMA = "fabric"` en la celda de configuración.
4. Ejecuta todas las celdas en orden.
5. Revisa la última celda de verificación.

Criterio de cierre: las cinco cifras de la celda de verificación dicen `OK` y ninguna celda falla. Si algo falla, anota la celda y el error.

Evidencia: captura de la celda de verificación y del historial de ejecución, y el mensaje de error completo si lo hubo.

P07 Notebook en Databricks

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no ejecutado.

Qué hacer:
1. Usa una prueba o una edición gratuita de Databricks (verifica disponibilidad).
2. Crea un catálogo `andina`, un esquema `datos` y dos volúmenes, `raw` y `bronze`. Sube los CSV a `raw`.
3. Importa el mismo notebook y cambia `PLATAFORMA = "databricks"` en la celda de configuración.
4. Ejecuta todas las celdas y revisa la verificación.

Criterio de cierre: las cinco cifras coinciden y las rutas de volúmenes funcionan.

Evidencia: captura de la verificación y del historial de ejecución.

P08 Optimización de la consulta en SQL Server

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: no medido en SQL Server. El experimento se hizo en SQLite con datos sintéticos.

Qué hacer:
1. Usa SQL Server (P03) con datos de prueba. Si no tienes 50 millones de pedidos, usa 5 millones y documenta la escala.
2. Ejecuta la consulta original y la reescrita de `sqlserver/optimizacion/02_optimizacion.sql` con `SET STATISTICS TIME ON` y `SET STATISTICS IO ON`, y con el plan de ejecución real activado.
3. Quita la conversión de tipo del cruce con tickets y mide otra vez (la ablación).
4. Compara resultados: deben ser idénticos.

Criterio de cierre: tiempos y planes medidos en SQL Server, con la escala documentada.

Evidencia: los mensajes de estadísticas, los planes de ejecución (archivo `.sqlplan`) y una tabla con los tiempos.

P09 Política de retención y contrato del proveedor de IA

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: la llamada a la API se ejecutó con dos modelos, con un costo de unos USD 0,03. Falta confirmar la política de retención de datos del proveedor.

Qué hacer:
1. Lee la política de privacidad y de retención de datos del proveedor (DeepSeek) y, si existe, el contrato de uso comercial.
2. Confirma los nombres de los modelos que se usaron y su precio vigente en su página oficial.
3. Anota si los textos enviados se usan para entrenar modelos y cuánto tiempo se conservan.

Criterio de cierre: hay un párrafo con la política citada textualmente, con su fecha de consulta y enlace.

Evidencia: captura o PDF de la política, con fecha, y el texto citado en `resultado.md`.

P10 Revisión de las 34 etiquetas de referencia

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: las etiquetas las propuso la IA y nadie las ha validado.

Qué hacer:
1. Abre `src/shared/tickets_ia/datos/etiquetas_reales.csv` (34 descripciones, con su tipo asignado).
2. Para cada fila, decide si la etiqueta es correcta. Si no lo es, anota el tipo correcto. Usa la taxonomía de `src/shared/tickets_ia/taxonomia.py` y las distinciones de la documentación (por ejemplo, `pedido_no_llega` frente a `entrega_no_recibida`).
3. Guarda el resultado en `docs/evidencia/P10/etiquetas_revisadas.csv`, con una columna `revisor` (rol, no nombre) y una columna `cambio` (sí/no).
4. Cuenta cuántas cambiaste.

Criterio de cierre: las 34 filas revisadas, con el conteo de cambios.

Evidencia: el CSV revisado y el número de cambios en `resultado.md`. Después, reemplaza la frase "PENDIENTE DEL AUTOR" en `docs/07_clasificacion_ia.md`.

P11 Muestra de tickets reales etiquetada por personas

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: el caso no trae textos nuevos para medir la clasificación de forma independiente.

Qué hacer:
1. Obtén tickets reales de otra fuente (exportación del sistema de soporte), anonimizados: sin nombres, documentos, teléfonos ni correos. Si no es posible anonimizarlos, no los envíes a ninguna API externa.
2. Toma una muestra estratificada por tipo, de 100 a 200 tickets.
3. Dos personas etiquetan cada ticket por separado. Calcula el acuerdo (kappa) entre ellas.
4. Compara con el modelo solo si la política de P09 lo permite.

Criterio de cierre: muestra etiquetada, acuerdo entre anotadores calculado y, si aplica, exactitud del modelo con intervalo de confianza.

Evidencia: la muestra anonimizada, el cálculo del acuerdo y el resultado en `resultado.md`.

P12 Definición de venta neta firmada y dueños por dominio

> **Revisión 01 nov 2026**
>
> La definición del caso ya está implementada. La firma y los responsables corresponden a adopción organizacional, no a una regla todavía indefinida.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: es una decisión de negocio. Hay que definir si la venta incluye devoluciones y a qué fecha se atribuye.

Qué hacer:
1. Reúne a finanzas y comercial. Decidan: qué cuenta como venta, si las devoluciones restan, a qué fecha se atribuye (compra o entrega) y quién es el dueño de cada dominio (ventas, pagos, clientes).
2. Registra la decisión en un documento de una página, con firmas de los roles (no de nombres personales en el repositorio).
3. Si la decisión cambia algo del modelo, abre un cambio en el repositorio con `--recalcular-gold`.

Criterio de cierre: documento firmado por los roles, guardado como PDF.

Evidencia: el PDF firmado (sin datos personales) y una nota en `resultado.md`.

P13 Fecha de entrega: límite del 21 de octubre frente al 31 de octubre

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: el enunciado fija el límite el 21 de octubre de 2026. El README dice que la entrega es del 31 de octubre. No hay prórroga escrita en los archivos.

Qué hacer:
1. Revisa el correo o el canal donde recibiste la prueba.
2. Si hubo prórroga, pide confirmación escrita al reclutador. Si no hubo, decide cómo presentarlo.

Criterio de cierre: confirmación escrita, citada en el README.

Evidencia: captura del correo de confirmación (sin datos personales de terceros).

P14 Licencias de Power BI Pro bajo F64 y costos de Fabric y Databricks

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: hipótesis sin cotización.

Qué hacer:
1. Revisa la documentación oficial de licencias de Power BI y de capacidades de Fabric. Confirma si quien consume informes sin capacidad F64 necesita Power BI Pro.
2. Pide cotización formal a Microsoft (o a un partner) para la capacidad que propones, y a Databricks para el consumo estimado.
3. Anota fecha, moneda y supuestos de cada cotización.

Criterio de cierre: dos cotizaciones con supuestos, o la confirmación de la regla de licencias con enlace oficial.

Evidencia: PDF de cotizaciones o captura de la página oficial con fecha.

P15 **Revisión visual de la presentación en un navegador**

> **Revisión 01 nov 2026**
>
> Esta versión reemplaza Mermaid y fuentes externas por SVG, CSS y fuentes del sistema. La validación visual de esta versión se registra en evidencia/presentacion.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: la sintaxis de los diagramas se validó con Mermaid 10.9.1. La maquetación se revisó solo en escritorio, con capturas de Microsoft Edge en modo sin cabeza. No se revisó en teléfono.

Qué hacer:
1. Abre `docs/presentacion_entrega.html` en Chrome o Edge (necesitas conexión para cargar las fuentes y Mermaid).
2. Recorre todas las pestañas en escritorio.
3. Abre las herramientas de desarrollador, activa la vista de teléfono (unos 400 px) y recorre las pestañas otra vez.
4. Anota lo que se corte, se superponga o no se lea.

Criterio de cierre: ninguna pestaña con texto cortado ni diagramas superpuestos en escritorio, y en teléfono sin desbordamiento horizontal de la página.

Evidencia: capturas de las pestañas con problemas y la lista de correcciones pendientes.

P16 Reporte de ventas contra SQL Server real

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: las pruebas usan una base en memoria, no un servidor.

Qué hacer:
1. Completa P03 y P04 (la tabla `gold.fact_venta_neta_diaria` y `gold.dim_canal` deben existir).
2. Crea un usuario de solo lectura en SQL Server.
3. Define la cadena de conexión en una variable de entorno de tu equipo, nunca en un archivo del repositorio. Ten instalado ODBC Driver 18 for SQL Server y `pyodbc`.
4. Ejecuta `python src/shared/reporte_ventas.py` para un día con ventas y compara con el valor calculado en DuckDB para ese mismo día.

Criterio de cierre: el reporte imprime el mismo total y el mismo número de pedidos que DuckDB para el día elegido, en los tres canales.

Evidencia: la salida del reporte, redactada (sin la cadena de conexión), y el cálculo de referencia.

P17 **Rotación de credenciales expuestas** —

> **Revisión 01 nov 2026**
>
> La exposición de la clave del proveedor está declarada en este documento, no verificada aquí. La contraseña del PR pertenece al enunciado: confirmar si existe en un sistema real antes de tratarla como credencial operativa. No se inspeccionaron secretos.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: urgente. Dos credenciales quedaron expuestas.

Qué hacer:
1. Clave de DeepSeek: genera una clave nueva en la consola del proveedor y revoca la anterior. Actualiza `.env.local` con la nueva. **No la pegues en ningún chat.**
2. Contraseña de administrador del pull request: cámbiala en el servidor. Revisa el historial del código; si el repositorio se sube a un servidor, borra el historial que la contenga o trata la contraseña como comprometida.
3. Revisa que `.env.local` no esté en ningún lugar que se suba a git (el `.gitignore` ya lo excluye, pero solo tiene efecto cuando el repositorio sea git).

Criterio de cierre: las dos credenciales anteriores ya no funcionan, y la nueva está solo en `.env.local` o en un gestor de secretos.

Evidencia: confirmación de revocación (captura sin mostrar la clave) y la fecha.

P18 **Umbrales de cuarentena y de volumen** —

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: el método está propuesto, pero los números no existen todavía. Con un solo archivo no hay histórico de treinta días.

Qué hacer:
1. Para calcular un umbral necesitas cargas diarias reales durante al menos treinta días: la tasa de registros en cuarentena por carga y el volumen de pedidos por día.
2. Con esos datos, calcula el promedio y la desviación estándar, y propón un umbral (por ejemplo, promedio más dos desviaciones).
3. La dirección y el dueño del dato aprueban el umbral.

Criterio de cierre: serie de treinta días, cálculo y aprobación escrita.

Evidencia: la serie y el cálculo, y la aprobación.

P19 Salidas guardadas de las ejecuciones declaradas

> **Revisión 01 nov 2026**
>
> Esta revisión registra una ejecución local de pytest y una consulta de solo lectura; no vuelve a ejecutar la carga destructiva desde cero ni Spark.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: el repositorio no conserva salidas de Spark SQL ni de las reglas en PySpark. Hay que dejarlas guardadas.

Qué hacer: ejecuta y guarda la salida de cada comando de este documento, con la fecha. Como mínimo:
- `python -m pytest -v > docs/evidencia/P19/pytest_completo.txt`
- `python scripts/cargar_datos.py --desde-cero > docs/evidencia/P19/carga_desde_cero.txt`
- `python scripts/cargar_datos.py > docs/evidencia/P19/carga_segunda.txt`

Criterio de cierre: los tres archivos existen y muestran `CARGA CORRECTA` y el resultado de las pruebas.

Evidencia: los archivos de salida.

P20 **Cifra mensual de APP**

> **Revisión 01 nov 2026**
>
> Calculado en Gold en esta revisión: APP junio 2026 = 376.510.600; septiembre = 448.880.300; variación +19,22 %. Evidencia mensual adjunta.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: la afirmación "APP crece" se retiró del texto porque no tiene cifra en la entrega.

Qué hacer: con `data/warehouse.duckdb`, ejecuta una consulta que sume la venta neta por mes y canal, y compara APP entre meses. Puedes usar:
`SELECT date_trunc('month', fecha) AS mes, canal, sum(venta_neta) AS venta FROM gold_venta_neta_diaria GROUP BY 1, 2 ORDER BY 1, 2;`

Criterio de cierre: tabla mensual de APP, y si la afirmación se restituye, con la cifra.

Evidencia: la tabla exportada y la decisión en `resultado.md`.

P21 **Prueba de la caída de TIENDA con datos de 2025 (recomendada)**

> **Revisión 01 nov 2026**
>
> Calculado en Gold: TIENDA junio–septiembre 2025 +59,79 %; 2026 −51,37 %. El patrón no se repite. Dos años no permiten una conclusión causal sobre estacionalidad.

INVENTARIO ORIGINAL · ESTADO NO ACTUALIZADO AUTOMÁTICAMENTE

Estado actual: la caída de TIENDA entre junio y septiembre de 2026 es un hallazgo. No sabemos si es estacional. Los datos de 2025 permiten comprobarlo.

Qué hacer: con la misma consulta de P20, compara TIENDA de junio a septiembre de 2025 contra junio a septiembre de 2026.

Criterio de cierre: la comparación con las cifras, y una conclusión que diga si la caída es o no es estacional según estos datos.

Evidencia: la tabla de comparación.

D01 **Completar Silver → modelo estrella → consumo**

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18

El ETL escribe silver_* y gold_venta_neta_diaria en DuckDB; DAX espera fact_pedido y el reporte usa gold.fact_venta_neta_diaria + dim_canal. El DDL por sí solo no carga datos.

CRITERIO DE CIERRE PROPUESTO

Implementar carga de dimensiones/hechos y claves, cliente desconocido y fechas; conciliar montos, pedidos y grano contra Silver.

Responsable sugerido: Ingeniería + BI

Evidencia: src/shared/andina_etl/store.py · sqlserver/ddl_tsql.sql · fabric/semantic_model/medidas.dax. No equivale a un fallo reproducido salvo que se indique expresamente.

D02 Borrados y correcciones fuera de ventana

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
Un pedido borrado del archivo sigue en Silver. Un pago corregido con fecha_carga antigua puede no releerse.	Contrato de updated_at y borrados/CDC; reconciliación completa periódica y pruebas de corrección tardía.

Responsable sugerido: Ingeniería + dueño de la fuente

Evidencia: README_ETL.md · pipeline.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D03 Revalidar relaciones cuando cambia el maestro

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
flag_cliente_desconocido solo se actualiza cuando se escribe el pedido. La cuarentena conserva historial sin un flujo explícito de resolución operativa.	Revalidación por cambios de dimensión; estado, responsable y fecha de resolución de cada incidencia.

Responsable sugerido: Ingeniería + calidad

Evidencia: README_ETL.md · store.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D04 Contratos de esquema y capacidad de ingesta

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
El flujo local materializa snapshots en pandas y compara contra tablas existentes. El hash del notebook usa binaryFile para el archivo completo.	Validar columnas/tipos antes de procesar; probar cambios de esquema, lotes grandes y memoria; extraer incrementos reales.

Responsable sugerido: Ingeniería

Evidencia: bronze.py · store.py · src/shared/spark/build_notebook.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D05 Recuperación Delta y ejecuciones concurrentes

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
El notebook no confirma todas las tablas como una sola transacción. No se evidencia coordinación de corridas ni prueba de concurrencia.	Publicar por versión validada, exclusión de corridas, reintentos controlados y prueba de fallo entre cada etapa.

Responsable sugerido: Plataforma

Evidencia: README_ETL.md · notebook Delta. No equivale a un fallo reproducido salvo que se indique expresamente.

D06 Automatizar calidad, entrega y observabilidad

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
Hay pruebas y log local, pero no se encontraron workflows CI/CD, programador ni alertas implementadas en los archivos del proyecto.	Pipeline de pruebas y despliegue, artefactos por versión, frescura y conciliación; simular un fallo que bloquee publicación y avise.

Responsable sugerido: Plataforma + líder técnico

Evidencia: tests/ · pipeline.py · docs/04_arquitectura.html. No equivale a un fallo reproducido salvo que se indique expresamente.

D07 Respaldo, retención, permisos y recuperación

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
El diseño propone seguridad, pero no aporta evidencia de permisos, RLS, restauración o retención de Bronze/cuarentena.	Acordar RPO/RTO, ensayar restauración y probar permisos mínimos; definir retención y borrado de datos personales.

Responsable sugerido: Plataforma + seguridad

Evidencia: docs/04_arquitectura.html · bronze.py · store.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D08 Meses sin ventas y contrato entre dialectos

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
LAG compara meses con filas. Si falta un mes, el “anterior” puede no ser el mes calendario previo; hoy sí existen los 21 meses en cada canal.	Calendario completo por canal y pruebas de cero ventas, cero denominador, fin de mes y paridad entre SQL/DAX/Gold.

Responsable sugerido: Analítica + BI

Evidencia: duckdb/01_ventas_recompra.sql · docs/01_sql_lectura.md. No equivale a un fallo reproducido salvo que se indique expresamente.

D09 Integrar soporte y cerrar el circuito operativo

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
Tickets ya se cargan opcionalmente en el ETL local. Las 99 fechas de cierre incoherentes y los 59 cierres sin fecha solo tienen tratamiento propuesto. El envío de reporte sigue como TODO.	Pipeline de soporte con reglas y SLA validado; integrar clasificación y revisión; implementar entrega del reporte con trazabilidad.

Responsable sugerido: Ingeniería + soporte

Evidencia: docs/02_catalogo_calidad.md · reporte_ventas.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D10 Endurecer el cliente LLM

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
interpretar acepta confianza numérica sin limitarla a [0,1]; el bucle reintenta JSON inválido, pero no gestiona explícitamente excepciones de transporte. La caché es de proceso.	Validar rango y valores finitos; manejar timeout/429 con presupuesto, backoff y cola de revisión; versionar prompt/modelo/resultados.

Responsable sugerido: Ingeniería IA

Evidencia: src/shared/tickets_ia/llm.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D11 Acordar contrato monetario y catálogos

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18	CRITERIO DE CIERRE PROPUESTO
El ETL convierte importes a enteros: falta explicitar cómo tratar fracciones. Categoría se limpia como texto sin un catálogo gobernado.	Definir precisión y redondeo o rechazo; probar importes decimales y catálogo desconocido; acordar evolución de canales/categorías.

Responsable sugerido: Finanzas + ingeniería

Evidencia: src/shared/andina_etl/rules.py. No equivale a un fallo reproducido salvo que se indique expresamente.

D12 **Empaquetar una entrega reproducible** —

OBSERVACIÓN ORIGINAL · ANTERIOR A A01–A18

La carpeta no contiene .git. Los documentos tienen estados y fechas divergentes; las reglas y definiciones se repiten entre motores.

CRITERIO DE CIERRE PROPUESTO

Repositorio o ZIP limpio con versión, exclusión de entornos/datos/secretos, comandos y evidencias; actualizar declaración de asistencia IA para esta presentación.

Responsable sugerido: Autor + líder técnico

Evidencia: README.md · CLAUDE.md · .gitignore. No equivale a un fallo reproducido salvo que se indique expresamente.

Alcance de la revisión

Se leyeron el enunciado, README, documentación técnica, pipeline, reglas, almacenamiento, Bronze, SQL, DDL, medidas DAX, reporte, cliente LLM y pruebas. Se consultaron agregados locales y se ejecutó pytest. No se desplegó infraestructura ni se llamaron APIs de IA. Las observaciones de producción son criterios propuestos, no una certificación exhaustiva.

Presentación rediseñada con asistencia de Codex. Se mantiene la autoría de la solución y la declaración previa de uso de Claude del README. Datos incrustados: agregados comerciales; no se incorporan nombres, documentos ni secretos.

Abrir inventario original ↗ · Abrir README ↗ · Evidencia de revisión ↗